

Feature Modules

▼ Relevant source files

Frontend/STUDER/src/app/features/users/user.model.ts

Frontend/STUDER/src/app/features/users/user.service.ts

Frontend/STUDER/src/assets/i18n/en.json

Frontend/STUDER/src/assets/i18n/es.json

This section provides a high-level overview of the functional modules within the STUDER frontend. Each feature is encapsulated within the `src/app/features/` directory and follows a consistent architectural pattern: **Model Interfaces** (defining the data contract), **Angular Services** (handling HTTP communication and business logic), and a **Component Tree** (managing the UI and user interaction).

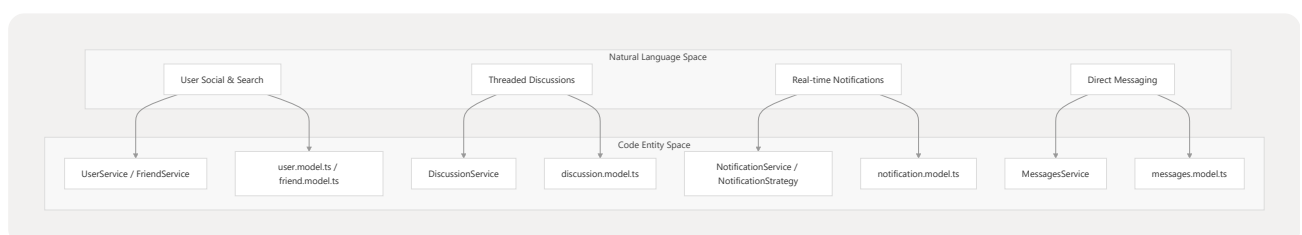
Feature Architecture Pattern

The frontend follows a modular structure where each domain feature is self-contained. Services leverage the `HttpClient` to interact with the Spring Boot backend, while components use reactive streams to manage state and display data.

System-Wide Feature Mapping

The following diagram maps high-level feature names to their primary code entities within the Angular application.

Feature to Code Mapping



Sources: Frontend/STUDER/src/app/features/users/user.service.ts 1-38

Frontend/STUDER/src/app/features/users/user.model.ts 1-60

Discussions Feature

The Discussions module allows users to engage in threaded conversations, categorized by tags and sorted by activity. It supports creating new topics, replying to existing messages, and marking discussions as favorites.

- **Key Logic:** Managed by `DiscussionService`, which handles paginated fetching of public and private discussions.
- **State Management:** Tracks "closed" states where new replies are prohibited.
- **Ordering:** Discussions are ordered by the `lastActivity` timestamp to keep active threads at the top.

For details, see [Discussions Feature](#).

Messaging Feature

The Messaging module implements a friend-gated direct chat system. To prevent spam, users can only initiate a full chat if they are confirmed friends; otherwise, the first message acts as a "Message Request."

- **Components:** `MessagesComponent` serves as the main hub for chat previews and active conversation windows.
- **Data Models:** Uses `ChatPreviewDTO` for the sidebar list and `DirectMessageResponseDTO` for the message history.
- **UI States:** Handles specific views for "Request Notice" (viewing a pending request) and "Send Placeholder" (initial contact).

For details, see [Messaging Feature](#).

Notifications Feature

STUDER employs a **Strategy Pattern** for notifications to handle different interaction types (System, Discussion, Message, and User) with unique navigation and display logic.

- **Strategy Factory:** The `NotificationStrategyFactory` determines which `NotificationStrategy` to apply based on the `LinkedType` of the notification.
- **Concrete Strategies:**
 - `DiscussionNotificationStrategy` : Navigates to the specific discussion thread.
 - `MessageNotificationStrategy` : Redirects to the relevant chat.
 - `UserNotificationStrategy` : Opens the profile of the interacting user.
- **Integration:** The `NotificationService` aggregates these strategies to provide a unified API for the `NotificationPanelComponent`.

For details, see [Notifications Feature](#).

User Profiles & Social Features

This module manages user identity, social connections (following/friends), and the global search functionality.

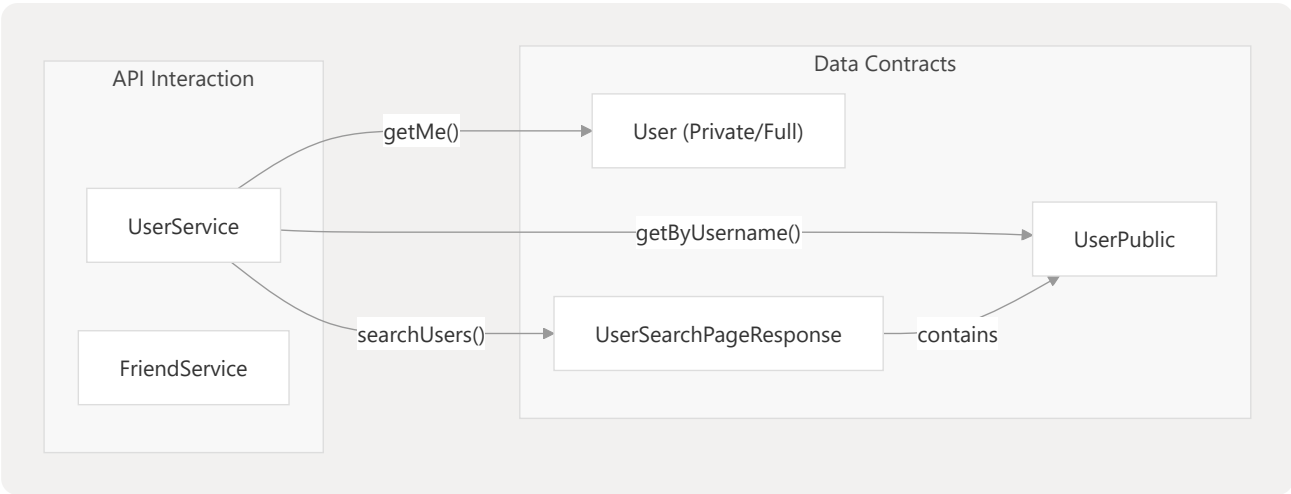
- **Profile Management:** `ProfileComponent` allows users to update their own data, including multipart file uploads for profile pictures via `UserService.updateMe`
`Frontend/STUDER/src/app/features/users/user.service.ts` 29-36
- **Social Graph:** `FriendService` manages the transition between "Following" and "Friends" (mutual following).
- **Search:** The `SearchComponent` utilizes `UserService.searchUsers` to find other participants by username or name `Frontend/STUDER/src/app/features/users/user.service.ts` 24-27

For details, see [User Profiles & Social Features](#).

User Data Flow

This diagram illustrates how user data moves from the service layer to the models used across these features.

User Data Entity Flow



Sources: Frontend/STUDER/src/app/features/users/user.service.ts 16-27
Frontend/STUDER/src/app/features/users/user.model.ts 32-60

Feature Localization

All feature modules are fully localized using the `ngx-translate` library. Translation keys are organized by feature within the `i18n` JSON files.

Feature	Translation Namespace	Key Examples
Search	search	search.placeholder , search.empty
Messages	messages	messages.request_notice , messages.send
Profile	profile	profile.change_photo , profile.follow
Discussions	discussions	discussions.create_title , discussions.closed

Sources: Frontend/STUDER/src/assets/i18n/en.json 101-173
Frontend/STUDER/src/assets/i18n/es.json 101-173